

July 01, 2024

Web Application Security Report

Production - API
Collation.AI

Contents

1. Engagement Summary 3

 1.1. Engagement Scope 3

 1.2. Engagement Purpose 4

2. Executive Summary 5

 2.1. Overview 5

 2.2. Effective Security 5

 2.3. Recommendations Summary 5

3. Technical Summary 6

 3.1. Summary of Findings 6

4. Findings 7

 4.1. Anti-caching headers not set to prevent caching of sensitive data 7

 4.2. X-Content-Type-Options header missing or invalid 9

 4.3. Content disposition missing for APIs 11

5. Conclusion 13

CONFIDENTIAL

1. Engagement Summary

1.1. Engagement Scope

Konfirmity Pte. Ltd. was engaged by Collation.AI to perform a Security Test to identify threats and risks against their application. The outcome of this engagement is for Collation.AI to understand the priority risks associated with these assets and how to mitigate them to minimize the likelihood and impact of a security breach.

The applications within the scope of this test were:

- api.collation.ai

CONFIDENTIAL

1.2. Engagement Purpose

Collation.AI would like to perform a penetration test in an attempt to gain unauthorized access to the targets defined within the scope with the main goals of understanding the risk associated with threat scenarios that lead to the following business impacts:

- **Financial Impact:** The target may be compromised or manipulated to trigger a financial impact to the organization through loss of revenue, security breach investigations, remediation costs, stolen assets or monetary funds and fines or legal action resulting from breaches of regulatory or legal requirements.
- **Reputation Impact:** The target may be compromised or manipulated to impact the reputation of the organization through brand damage, loss of clients and loyalty, reduced acquisition of products and services, reduction in share price and capital, a less attractive option for employees and the corresponding disaster recovery and public relations costs.
- **Compliance Impact:** If the target is required to comply with one or more industry standards or regulations, domestic and international legal requirements, or internal policies, then a security breach may lead to compliance fines, increased auditing frequency and stricter audit requirements, additional security controls and projects, as well as loss of regulatory licenses, or legal action and data breach notification fines. Common standards and regulations include PCI DSS, GDPR, HIPPA, ASIC, Australian Privacy Principals, ASD, NIST, and WHS.
- **Privacy Impact:** If the target contains sensitive, confidential or private information, then a security breach may lead to a direct impact to customers - including financial losses through credit card breaches, mental health impacts from private or health related information breaches, identity theft through password and online account breaches, and physical risks through personally identifiable information being leaked. This can cause a negative impact on finances, reputation and compliance - as detailed above.

2. Executive Summary

2.1. Overview

Konfirmity Pte. Ltd. performed a Security Test against Collation.AI's Production - API application to identify security flaws and corresponding risks to the business. This project was completed on July 01, 2024. The primary aim of a penetration test is to identify and validate security weaknesses that exist within an existing system or application. Penetration Testing allows missing security controls to be identified, risks to the organization calculated, and mitigation controls implemented to proactively reduce this risk. The following table summarizes the issues identified:

Description	Critical	High	Medium	Low	Info	Total
Web Application Assessment	0	0	1	0	2	3

2.2. Effective Security

Konfirmity Pte. Ltd. notes the following control areas were found to be in place and effective against motivated and skilled attack:

- Cookies are set with 'HttpOnly', 'Secure', 'SameSite' attributes, '__Host-' prefix, and appropriate path settings, and do not expose sensitive information.
- Subresource Integrity (SRI) checks are implemented correctly for external JavaScript, CSS, and stylesheet assets.
- HTTP Strict Transport Security (HSTS) header is enabled and correctly configured.
- Content Security Policy (CSPv2) is correctly configured and applied.
- Anti-clickjacking protections (e.g., X-Frame-Options headers) are implemented effectively.
- CSRF protection is implemented for all form submissions.
- The application fails securely, and debug mode is disabled in both application and server environments.
- Sensitive data is not included in query string parameters.
- Server information is not disclosed in HTTP headers or error messages.
- Session tokens are secure: they do not appear in URL parameters or error messages, and contain at least 64 bits of entropy.
- No usage of eval() was detected, preventing potential eval injection risks.
- Output encoding maintains user's character set and locale, ensuring safe Unicode handling.
- All communications are secured, avoiding plaintext HTTP requests.
- Content type character set is specified, and Referrer-Policy header is set and configured correctly.
- Password hints or knowledge-based questions are not used for authentication.

2.3. Recommendations Summary

Each of the security flaws detailed in this report has independent mitigation controls to best protect the solution and supporting systems. A high-level summary of mitigation steps is listed below:

- Set appropriate anti-caching headers (e.g., 'Cache-Control: no-store, no-cache, must-revalidate' and 'Pragma: no-cache') to prevent caching of sensitive data by browsers and intermediaries.
- Ensure the X-Content-Type-Options header is correctly set to 'nosniff' to prevent browsers from interpreting files as a different MIME type.
- Include the 'Content-Disposition' header for API responses to specify how content should be handled by the browser, enhancing security by preventing unintended content exposure or download.

3. Technical Summary

3.1. Summary of Findings

Ref.	Vulnerability Summary	Risk
4.1	Anti-caching headers not set to prevent caching of sensitive data The application should set sufficient anti-caching headers to ensure that sensitive data is not stored in the cache of modern browsers.	MEDIUM Resolved 0 / 2
4.2	X-Content-Type-Options header missing or invalid The X-Content-Type-Options header is not implemented. This header prevents MIME type sniffing which can lead to security vulnerabilities.	INFO Resolved 0 / 2
4.3	Content disposition missing for APIs API responses should include the Content-Disposition header with the 'attachment' directive and a filename, to define the default filename for the downloaded file.	INFO Resolved 0 / 2

4. Findings

4.1. Anti-caching headers not set to prevent caching of sensitive data

MEDIUM

6.9

Resolved

0 / 2

ASVS 8.2.1

CWE-525

WASC-13

Details

The security assessment revealed that the application does not consistently set anti-caching headers for responses containing sensitive data. Anti-caching headers are crucial in controlling the caching behavior of browsers and intermediate proxies to prevent them from storing copies of sensitive information. This oversight can lead to unauthorized access to sensitive data through browser caches or shared systems where multiple users might access the same device.

Impact

If sensitive information such as personal details, financial information, or authentication data is cached, it can be accessed by unauthorized users especially on shared or public systems. This could lead to data breaches, identity theft, or unauthorized transactions. Ensuring that sensitive data is not cached is critical for maintaining the confidentiality and integrity of user data.

Recommendations

Konfirmity Pte. Ltd. recommends Collation.AI consider the following:

- **Set Proper HTTP Headers:** Configure your web server to include specific HTTP headers that instruct the browser and any intermediate caches not to store the content of the responses. Examples of such headers include:

```
Cache-Control: no-store, no-cache, must-revalidate, proxy-revalidate
Pragma: no-cache
Expires: 0
```

These headers should be included in all responses that serve sensitive data.

- **Configure Web Frameworks:** Ensure that your web application framework is configured to automatically add these headers to responses containing sensitive data. Most modern frameworks provide easy ways to set default headers or intercept responses to modify headers.

For example, in a Spring Boot application:

```
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import javax.servlet.http.HttpServletResponse;

@Controller
public class SecureController {
    @GetMapping("/sensitive-data")
    public String handleSensitiveData(HttpServletResponse response) {
        response.setHeader("Cache-Control", "no-store, no-cache, must-revalidate, proxy-revalidate");
        response.setHeader("Pragma", "no-cache");
        response.setHeader("Expires", "0");
        return "sensitive-data";
    }
}
```

- **Regular Audits and Tests:** Conduct regular security audits and penetration testing to verify that caching headers are correctly set across all endpoints that handle sensitive data. This ensures that there are no regressions or overlooked endpoints as the application evolves.

- Client-Side Controls: While server-side headers are crucial, ensure that client-side JavaScript is also designed to not store sensitive data in local storage or session storage. Educate developers about the risks associated with improper data storage on the client side.
- Documentation and Policies: Document your caching policies and header configurations as part of your organization’s security policies. Ensure that all new developers are trained on these policies to maintain consistency and awareness across the team.

Related Links

- https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/04-Authentication_Testing/06-Testing_for_Browser_Cache_Weaknesses

Occurrences

Occurrence # 1		NEW
GET https://api.collation.ai/v1/9759a712-8b5e-433e-8589-dfa11bd502a4/classifications/?page=1&per_page=20&sort_column=created_at&sort_direction=desc		
First Seen on July 1, 2024 8:20 PM.		Updated on July 1, 2024 8:20 PM
Occurrence # 2		NEW
GET https://api.collation.ai/v1/9759a712-8b5e-433e-8589-dfa11bd502a4/currency_accounts/?page=1&per_page=20&sort_column=created_at&sort_direction=desc		
First Seen on July 1, 2024 8:20 PM.		Updated on July 1, 2024 8:20 PM

4.2. X-Content-Type-Options header missing or invalid

INFO

Resolved 0 / 2

ASVS 14.4.4

CWE-693

WASC-15

Details

The security assessment revealed that the `X-Content-Type-Options` HTTP header is missing or improperly configured on the web application's responses. This header is essential as it instructs the browser to adhere strictly to the declared content type in the `Content-Type` header, without attempting to guess the MIME type (a process known as MIME type sniffing). The lack of this header can expose the application to various security risks, particularly where content type handling is concerned.

Impact

Without the `X-Content-Type-Options` header set to `nosniff`, browsers might attempt to interpret files with ambiguous extensions or MIME types as a different MIME type than what is specified by the server. This can lead to security vulnerabilities such as Cross-Site Scripting (XSS) if the browser executes files as scripts when they are not intended to be. This misinterpretation can particularly affect older browsers more significantly, but it remains a concern across all browser types.

Recommendations

Konfirmity Pte. Ltd. recommends Collation.AI consider the following:

- Implement X-Content-Type-Options Header: Ensure that the `X-Content-Type-Options: nosniff` header is included in all HTTP responses that serve content. This header should be universally applied to both static and dynamic content to prevent MIME sniffing by the browser. Example for Apache configuration:

```
Header always set X-Content-Type-Options "nosniff"
```

Example for Nginx:

```
add_header X-Content-Type-Options "nosniff" always;
```

- Verify Content-Type Handling: Ensure that all content served by the application has a correctly defined `Content-Type` header. This practice helps in reducing the risk of MIME type confusion and enhances overall security posture.
- Regular Audits of HTTP Headers: Conduct periodic audits of your application's HTTP headers to ensure that security headers like `X-Content-Type-Options` are consistently present and correctly configured. This can be integrated into regular security reviews or automated using tools that scan for missing headers.
- Educational Initiatives for Developers: Organize training sessions for developers on the importance of proper HTTP header management and security best practices. Understanding the impact of headers like `X-Content-Type-Options` can lead to more secure coding practices.
- Adopt a Content Security Policy (CSP): While CSP is primarily used for specifying legitimate sources of resources, it can also help mitigate certain types of attacks including XSS. Implementing CSP can serve as an additional layer of defense in depth, complementing headers like `X-Content-Type-Options`.

Related Links

- <https://owasp.org/www-project-secure-headers/>

Occurrences

Occurrence # 1

NEW

GET https://api.collation.ai/v1/9759a712-8b5e-433e-8589-dfa11bd502a4/currency_accounts/?page=1&per_page=20&sort_column=created_at&sort_direction=desc

First Seen on July 1, 2024 8:20 PM.

Updated on July 1, 2024 8:20 PM

Occurrence # 2

NEW

GET https://api.collation.ai/v1/9759a712-8b5e-433e-8589-dfa11bd502a4/classifications/?page=1&per_page=20&sort_column=created_at&sort_direction=desc

First Seen on July 1, 2024 8:20 PM.

Updated on July 1, 2024 8:20 PM

CONFIDENTIAL

4.3. Content disposition missing for APIs

INFO

Resolved 0 / 2

ASVS 14.4.2

CWE-116

Details

The security assessment revealed that the Content-Disposition header is missing in API responses where files are expected to be downloaded. The Content-Disposition header is crucial for indicating if the content should be treated as an inline entity or as an attachment that can be downloaded and saved locally. This header also specifies the default filename for the downloaded file, enhancing the user experience by preventing renaming issues and reducing confusion.

Impact

Without the proper use of the Content-Disposition header, users might experience inconsistent behavior across different browsers when attempting to download files from the API. This can lead to usability issues and potentially expose the system to security risks if the content is not handled correctly by the browser. For instance, files intended to be downloaded could be executed in the browser context, leading to security vulnerabilities such as Cross-Site Scripting (XSS) or other types of code injection attacks.

Recommendations

Konfirmity Pte. Ltd. recommends Collation.AI consider the following:

- Implement Content-Disposition in API Responses: Ensure that API responses that prompt a file download include the Content-Disposition header with the attachment directive, and specify a default filename. This practice prevents the browser from mistakenly displaying the content inline. Example:

```
Content-Disposition: attachment; filename="example.pdf"
```

- Validate the Filename: Make sure that the filename specified in the Content-Disposition header is properly validated and sanitized to avoid path traversal or file inclusion vulnerabilities. Avoid using user input directly in the filename without proper sanitization.
- Content Type Handling: Alongside Content-Disposition, ensure that the Content-Type header is appropriately set according to the type of the file being served. This helps the browser understand how to process the file. Example:

```
Content-Type: application/pdf
```

- Regular Audits and Testing: Regularly audit and test API endpoints to ensure that headers like Content-Disposition are correctly implemented and function as expected. Consider automated testing tools to assist in this process.
- Security Awareness and Best Practices Training: Educate your development team about the importance of proper header implementation in API responses, particularly for content delivery and security headers. Regular training can help mitigate security risks associated with improper API response handling.

Occurrences

Occurrence # 1

UNRESOLVED

```
GET https://api.collation.ai/v1/9759a712-8b5e-433e-8589-dfa11bd502a4/classifications/?
page=1&per_page=20&sort_column=created_at&sort_direction=desc
```

First Seen on July 1, 2024 8:15 PM.

Updated on July 1, 2024 8:20 PM

Evidence:

No Content-Disposition header found.

Occurrence # 2

UNRESOLVED

GET https://api.collation.ai/v1/9759a712-8b5e-433e-8589-dfa11bd502a4/currency_accounts/?page=1&per_page=20&sort_column=created_at&sort_direction=desc

First Seen on July 1, 2024 8:15 PM.

Updated on July 1, 2024 8:20 PM

Evidence:

No Content-Disposition header found.

CONFIDENTIAL

5. Conclusion

The web application has several critical security vulnerabilities that need immediate attention. Anti-caching headers are not properly set, the X-Content-Type-Options header is missing or invalid, and content disposition is missing for APIs. These issues can lead to sensitive data exposure and potential security breaches. While some security measures like setting cookies without 'HttpOnly' and 'Secure' attributes are in place, there are still significant gaps in security controls. It is crucial to address these vulnerabilities promptly to prevent exploitation by malicious actors.

CONFIDENTIAL